# FloorSet-Prime Solver: Code Modules and Reproduction Guide

Team ICB, 2026-10-25. Markdown copy of the shared doc of the same name.

## Overview

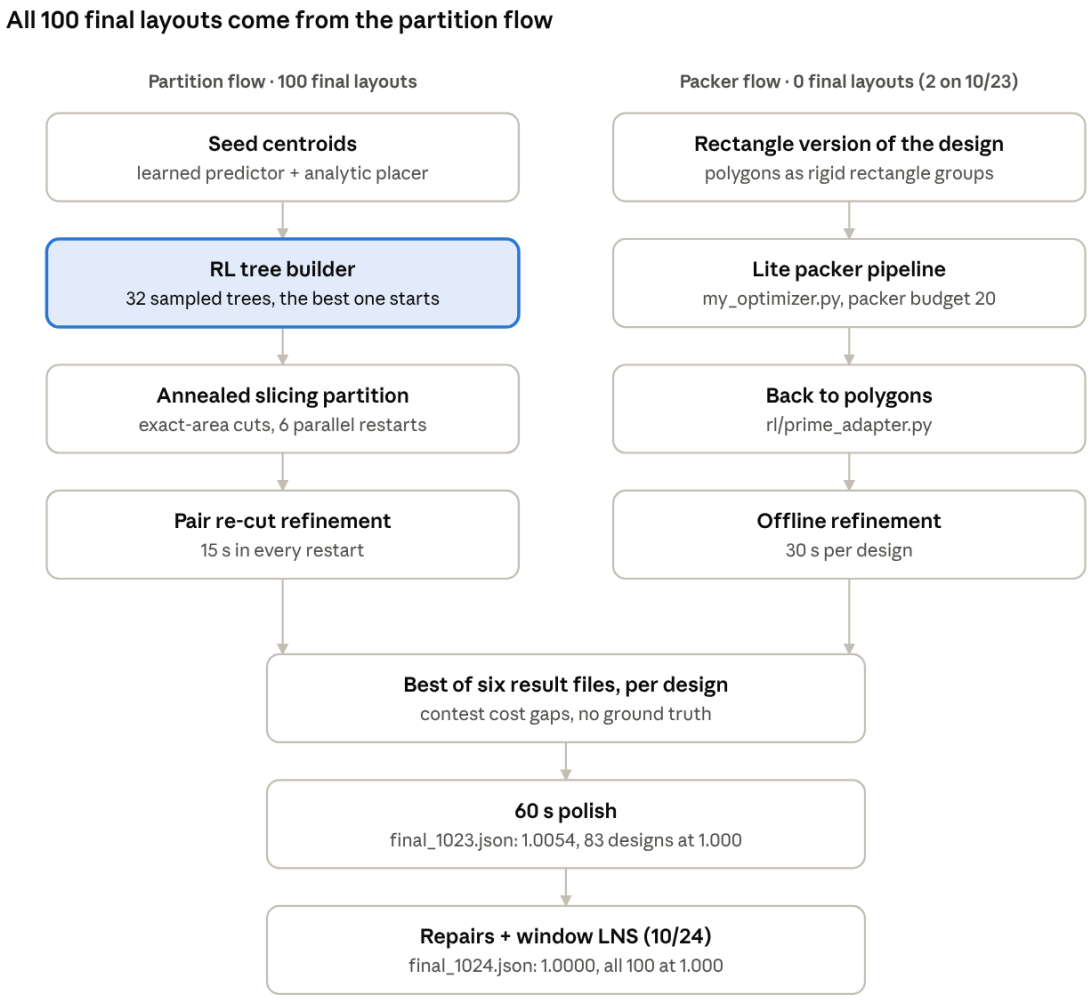
The Team ICB FloorSet-Prime solver scores a weighted quality of 1.0000 on the 100 Prime test designs (ground truth = 1.000). All 100 layouts are valid, all 100 score exactly 1.000, and all 100 have lower wirelength (HPWL) than the ground truth, on average 4.2% lower (`final_1025.json`). On 10/23 the figures were 1.0054, with 83 designs at 1.000. A full rebuild from scratch on 10/24–25 reached 1.0000 with all 100 at 1.000 and no hand-run steps.

Prime blocks are rectilinear polygons, and every ground-truth layout tiles its frame with zero white space. The solver builds an exact-area slicing partition of the frame, starts it from a learned (RL) tree builder, refines it with local re-cuts, and keeps the best layout per design across several flows. A last stage (10/24) repairs the remaining constraint violations and shortens wirelength by re-partitioning small windows of the layout.

The code is on GitHub (branch `floorset_prime_rl` of `andyfox-rushc/FloorSet`) and in the archive `floorset_prime_1025.tar.gz`; see Getting the code. Both hold the source code (top-level Prime scripts, `rl/`, `scripts/`, `tests/`), the checkpoints the final flow loads, the per-flow result JSONs that the final best-of reads, the report PDFs, this file, and the 100 test designs (5 MB). The 44 GB training set isn't included in either; it's public.

## Pipeline

Two independent flows produce candidate layouts. A best-of step then keeps the cheapest valid layout per design, choosing without the ground truth, and a 60 s polish refines it. A last stage (10/24) repairs the soft violations left and shortens wirelength by re-partitioning small windows. In the final result all 100 layouts come from the partition flow, most of them started by the RL tree builder; the packer flow won designs 8 and 76 on 10/23, and on 10/24 both went to zero-white-space partition layouts, which the last stage made cheaper.




1. **Partition flow** (`prime_partition.py` → `rl/partition.py`). The frame is the pin bounding box, which matches the ground-truth frame on all 100 designs, so its area equals the total block area. Rigid blocks (fixed-shape and MIB groups) go near seed centroids from the learned centroid predictor (`rl/prime_placer.py`) or the

analytic placer (`rl/analytic.py`). The free region is then split among the soft blocks by an annealed slicing tree with exact-area cuts. Six parallel restarts alternate learned and analytic seeds, with a time budget of 80 s × $(n/77)^2$.

2. **RL tree builder** (`rl/tree_policy.py`, `FLOORSET_TREE_POLICY`). A network trained with REINFORCE on the Prime training set builds the starting slicing tree top-down. 32 sampled trees are scored and the best one starts the anneal.
3. **Pair re-cut refinement** (`rl/refine.py`, `FLOORSET_REFINE=15`), run on every restart's layout. Moves: merge two touching soft blocks and re-split them with one straight exact-area cut, re-place a rigid block in a corner, re-split three blocks, and push or route-push blocks to fix edge-pin and cluster violations. Iterated local search adds random kicks.
4. **Packer flow** (`prime_optimizer.py` → `rl/prime_adapter.py` → `my_optimizer.py`). The Lite (rectangle) solver runs on a rectangle version of the design (polygons as rigid rectangle groups), and its output is converted back to polygons. It's a backup source of candidates; it gives no final layout since 10/24.
5. **Best-of and polish** (`scripts/final_polish.py`). Each design's layouts are compared across flows with `prime_evaluate.relative_cost`, using the contest cost gaps and no ground truth. The winner gets a 60 s refinement from a fresh seed, and the result is never worse than its input.
6. **Repairs and window LNS** (`scripts/lns_final.py` → `rl/window.py`, new 10/24). The partition annealer is re-run on a window of the layout, with every block outside it held fixed. Repair moves built on it fix what pair moves can't: an edge pin behind a band of blocks (window re-partition), a goal corner walled off by preplaced blocks (pocket swap), and a split cluster (cluster shift). A large-neighbourhood search (LNS) then re-anneals random windows of 6–14 blocks for 1 s each, runs a greedy descent, and keeps a result only if wirelength or violations drop. The descent skips moves whose blocks haven't changed since they last failed ("don't-look bits"). These settings (10/24, from `prime_results/improvement_proposals.md`) gave 3× the tries and 31% more wirelength gain in the same time than the first LNS (3 s anneals, 6–20 blocks, full descent). Each design starts from both the polished pick and its best zero-white-space layout.

## Code modules

All paths are relative to `iccad2026contest/` in the repo root (`FloorSet/` from GitHub, `FloorSet-Prime/` from the archive). Line counts are as of 10/24.

### Prime entry points

| File | Lines | Role |
|---|---|---|
| `prime_evaluate.py` | 324 | Contest cost on polygon layouts (overlap, area ±1%, corner cap 16, fixed/MIB shapes, edge pins, clusters), CLI runner, `relative_cost` pick rule |
| `prime_partition.py` | 88 | Partition-flow optimizer: seeds, then `rl/partition.solve_parallel` |
| `prime_optimizer.py` | 32 | Packer-flow optimizer: Lite pipeline on the rectangle version of the design |
| `prime_combined.py` | 56 | Solve-time best of partition and packer flows (not used for the final result; the best-of runs offline) |
| `prime_baseline.py` | 47 | Shelf-packing baseline (7.51), a harness check |
| `prime_grid.py` | 83 | Grid flow (RPolyPlan global placement + JigsawPlanner grid assignment); parked, rarely valid |

### rl/ modules used by the Prime flows

| File | Lines | Role |
|---|---|---|
| `rl/partition.py` | 874 | Zero-white-space slicing partitioner: problem set-up, pockets, annealed tree, parallel restarts, refine hook |
| `rl/partition_kernels.py` | 255 | numba kernels for the rectangle-set operations the partitioner spends its time in |
| `rl/refine.py` | 927 | Pair re-cut refinement, rigid moves, 3-block re-splits, pushes, route pushes and the corridor re-slice, iterated local search; calls the window repairs when those fail |
| `rl/window.py` | 611 | Window re-partition (the partitioner on part of a layout, the rest fixed): edge-pin window, pocket swap, cluster shift, large-neighbourhood search (new 10/24) |
| `rl/tree_policy.py` | 308 | RL slicing-tree builder (policy network, sampling, greedy decode) |
| `rl/prime_placer.py` | 66 | Learned centroid predictor for Prime seeds (network from `rl/learned_placer.py`) |
| `rl/polygon.py` | 185 | Rectilinear polygon geometry: rectangle decomposition, rotations, shape keys, de-duplication |
| `rl/prime_adapter.py` | 236 | Prime ↔ Lite conversion for the packer flow; polygon-exact metrics |
| `rl/kick_policy.py` | 186 | Learned kicks for the refiner (trained, lost to random kicks, not used) |
| `rl/global_alm.py` | 249 | Grid flow stage 1 (parked) |
| `rl/grid_assign.py` | 1021 | Grid flow stages 2–3 (parked) |

### Lite pipeline (packer flow and analytic seeds)

| File | Role |
|---|---|
| `my_optimizer.py` | Lite contest solver: candidates, packer, finish; also `_pmap` process pool |
| `rl/analytic.py` | Analytic global placement, learned-placer candidates, best-of |
| `rl/packer.py` | Sequence-pair packer with contour packing (`FLOORSET_PACK_CONTOUR`) |
| `rl/repack.py`, `rl/slicing.py` | Re-pack and slicing polish; `contest_relative_cost` |
| `rl/legalize.py`, `rl/sizing.py` | Constraint repair and shape fitting |
| `rl/learned_placer.py` | Learned placer network (also the backbone of `rl/prime_placer.py`) |

| File | Role |
|---|---|
| `rl/env.py`, `networks.py`, `encoder.py`, `ppo.py`, `finetune.py`, `train.py`, `ordering.py`, `reward.py`, `compaction.py`, `anneal.py`, `data.py` | AlphaChip-style RL placer (Lite "base" candidate) |

### Scripts

| Script | Role |
|---|---|
| `scripts/reproduce_final.sh` | Rebuilds the final result from scratch (new 10/23; LNS stage 10/24) |
| `scripts/verify_results.py` | Re-scores any result file from its stored layouts (new 10/23) |
| `scripts/final_polish.py` | Best-of over flows + 60 s refinement |
| `scripts/lns_final.py` | Final stage (new 10/24): repairs + window LNS in rounds on top of the polished result |
| `scripts/lns_designs.py`, `lns_batch_1024.sh` | LNS on one design, and the batch runner used for the 10/24 rounds |
| `scripts/merge_layout.py`, `refine_designs.py` | Put one layout into a result file only if it is valid and cheaper; refine chosen designs |
| `scripts/refine_offline.py` | Refines every layout in a result file (30 s each) |
| `scripts/merge_results.py` | Replaces rows of a result file with rows from a rerun |
| `scripts/eval_partial.py` | Scores a stopped run's `.partial` file |
| `scripts/prime_best_of.py` | Best of two result files, as `prime_combined.py` picks |
| `scripts/train_prime_placer.py` | Trains `checkpoints/prime_placer.pt` on `../PrimeTensorData` |
| `scripts/train_tree_policy.py` | Trains the RL tree builder (REINFORCE) |
| `scripts/train_tree_policy_anneal.py` | Tree-builder fine-tuning with an anneal reward (no held-out gain) |
| `scripts/compare_tree_policies.py` | Paired test of two tree-builder checkpoints on held-out designs |
| `scripts/train_kick_policy.py`, `build_layout_pool.py`, `eval_kick_policy.py` | Kick-policy training and evaluation |
| `scripts/generate_prime_report_1022.py` + `prime_report_1022_text.json` | Builds `prime_report_1022.pdf`; with `--notes scripts/prime_report_1024_text.json` (or `_1025_text.json`) and the final files as extra flows, `prime_report_1024.pdf` and `prime_report_1025.pdf` |
| `scripts/night_1021.sh`, `night_full_1021.sh`, `packer_ils_1021.sh`, `report_after_night.sh`, `after_tree_1022b.sh` | The run chains that produced the published result files |

Tests for the Prime code: `tests/test_prime_evaluate.py`, `tests/test_partition.py`, `tests/test_partition_kernels.py`, `tests/test_refine.py`, `tests/test_window.py` (new 10/24), `tests/test_prime_packer.py`.

## Checkpoints and data

The final flow loads six checkpoint files (about 30 MB together), all in `checkpoints/`. If a file is missing, its flow falls back to a non-learned path rather than failing.

| Checkpoint | Size | Loaded by | Trained by |
|---|---|---|---|
| `checkpoints/tree_policy_best_8559.pt` | 65 KB | `rl/partition.py` via `FLOORSET_TREE_POLICY` | `scripts/train_tree_policy.py` (10/21, Prime training set) |
| `checkpoints/prime_placer.pt` | 6.9 MB | `prime_partition.py` (learned seeds) | `scripts/train_prime_placer.py` (10/20–21, 320k designs, val L1 0.084) |
| `checkpoints/policy.pt` | 0.3 MB | `my_optimizer.py` (Lite RL placer) | `rl/train.py` (Lite, September) |
| `checkpoints/placer_a.pt` | 1.3 MB | `rl/analytic.py` learned candidate 1 | `scripts/train_placer.py` (Lite) |
| `checkpoints/placer_r.pt` | 7.7 MB | learned candidates 2–3, packer anchor | `scripts/train_placer.py` (Lite, relation head) |
| `checkpoints/placer_relB.pt` | 7.7 MB | learned candidate 4 | `scripts/train_placer.py` (Lite, relation head) |

The branch and the archive also include `kick_policy_v2.pt` and `tree_policy_anneal2.pt`, the two RL experiments that didn't beat their baselines; nothing in the final flow loads them.

The 100 test designs are in `PrimeTensorDataTest/` in the repo root (5 MB, included); the test loader downloads them on first use if missing. Training data, `PrimeTensorData/` in the repo root (44 GB), is needed only to retrain; it downloads through `primeLoader.py` as the repo README describes.

## Getting the code

Either source below has everything needed to run this guide. Every file the two have in common is byte-identical (checked 10/25).

| Source | Where | Creates | Size |
|---|---|---|---|
| GitHub | branch `floorset_prime_rl` of `https://github.com/andyfox-rushc/FloorSet` | `FloorSet/` | 128 MB download, 480 MB checked out |
| Archive | `floorset_prime_1025.tar.gz` | `FloorSet-Prime/` | 31.6 MB |

The branch is Intel's FloorSet repo (upstream commit `aadddcc`) plus one commit, "Prime Data set", that adds the Team ICB code, checkpoints, results, reports and the Prime test designs. It also keeps the upstream files the archive leaves out: the Lite test designs (`LiteTensorDataTest/`, 255 MB), images, notebooks and the contest PDF. The archive has 28 run logs (`logs/`, `prime_results/*.log`) that the branch doesn't.

From GitHub (public, so HTTPS needs no login):

```
git clone -b floorset_prime_rl --single-branch https://github.com/andyfox-rushc/FloorSet.git
cd FloorSet
```

With an SSH key registered on GitHub, `git@github.com:andyfox-rushc/FloorSet.git` works as well. Plain `git clone` without `-b` checks out `main`, which is upstream FloorSet without the Prime code.

From the archive:

```
tar xzf floorset_prime_1025.tar.gz          # creates FloorSet-Prime/
cd FloorSet-Prime
```

The name of the top folder doesn't matter: the code and scripts use only relative paths. Below, "the repo root" means `FloorSet/` or `FloorSet-Prime/`. Checked from a fresh clone of the branch on 10/25: the full test suite passed (288 passed, 3 skipped), the re-score in section 1 gave the expected output, and the partition flow solved design 0 at cost 1.000 in 28 s.

## Environment setup

Produced on an 8-core, 15 GB WSL2 Linux machine, CPU only. All commands run from `iccad2026contest/` in the repo root, with the venv at `../venv` (the repo root's `venv/`, which `scripts/reproduce_final.sh` expects).

| Package | Version |
|---|---|
| Python | 3.14.4 |
| torch | 2.14.0 (CPU use) |
| numpy | 2.5.2 |
| numba | 0.68.0 |
| scipy | 1.18.1 |
| shapely | 2.1.2 |

numba and scipy are needed by the Prime code only; the contest's `requirements.txt` doesn't list them. `requirements_prime_venv.txt` (in the repo root) is the full `pip freeze` of the venv.

```
# in the repo root (FloorSet/ or FloorSet-Prime/)
python3 -m venv venv
venv/bin/pip install -r requirements_prime_venv.txt
cd iccad2026contest
../venv/bin/python -m pytest -q tests/test_prime_evaluate.py tests/test_partition.py tests/test_partition_kernels.py tests/test_refine.py tests/test_window.py
```

The evaluator imports the dataset classes from the parent folder (`prime_dataset_test.py`, `utils.py`), so keep `iccad2026contest/` inside the repo root as the clone or the archive lays it out. A 6-restart partition run uses up to about 6 GB; set `FLOORSET_PARTITION_MEMO=20000` for long-lived processes or when sharing the machine.

## Reproducing the results

Scores are deterministic once a layout exists. Solving isn't: restarts run in parallel against wall-clock budgets, so a rebuild lands near the published numbers rather than on them. The last stage's window search is timed and random as well.

### 1. Re-score the published layouts (5 s)

```
../venv/bin/python scripts/verify_results.py prime_results/final_1025.json
```

Expected output (checked 10/25):

```
prime_results/final_1025.json: weighted score 1.0000 (without runtime term 1.0000) | feasible 100/100 | mean HPWL gap -0.042, area gap 0.000, V_rel 0.000
  at 1.000: 100 | HPWL under GT: 100 | within 5%: 100
```

`final_1024.json` still re-scores at 1.0000 (mean HPWL gap −0.031) and `final_1023.json` at 1.0054.

### 2. Rebuild everything from scratch (about 13 h)

```
bash scripts/reproduce_final.sh                                  # writes prime_results/repro/
IDS="0 49 98" bash scripts/reproduce_final.sh /tmp/repro_small    # a subset, for a quick check
LNS_ROUNDS=60,60 LNS_SEEDS=1,2 IDS="8 50 76" bash scripts/reproduce_final.sh /tmp/repro_small   # short LNS stage
```

Steps run in order; a step whose output exists is skipped, so a stopped run resumes. The script ends by re-scoring every file.

| Step | Output | Command (inside the script) | Published equivalent | Published score | Rebuild 10/25 | Wall clock |
|---|---|---|---|---|---|---|
| 1 | `packer.json` | `FLOORSET_SOLVE_RUNS=1 FLOORSET_PACK=20 prime_evaluate.py --evaluate prime_optimizer.py` | `contour_pack20.json` | 1.2247 | 1.2318 | 1 h 41 min |
| 2 | `packer_refined.json` | `scripts/refine_offline.py` (30 s per design) | `contour_pack20_ils.json` | 1.1425 | 1.1427 | 50 min |
| 3 | `partition.json` | `prime_evaluate.py --evaluate prime_partition.py` | `partition_full_t80b.json` | 1.2915 | 1.1953 | 2 h 13 min |
| 4 | `partition_refined.json` | `scripts/refine_offline.py` | `partition_t80b_refined.json` | 1.1462 | 1.0187 | 50 min |
| 5 | `partition_refine.json` | step 3 + `FLOORSET_REFINE=15` | `partition_full_refine.json` | 1.1229 | 1.0199 | 2 h 42 min |
| 6 | `partition_refine_tree.json` | step 5 + `FLOORSET_TREE_POLICY=checkpoints/tree_policy_best_8559.pt` | `partition_full_refine_tree_merged.json` | 1.1077 | 1.0071 | 2 h 43 min |
| 7 | `final.json` | `scripts/final_polish.py --flow ... --seconds 60 --jobs 3` | `final_1023.json` | 1.0054 | 1.0016 | 34 min |
| 8 | `final_lns.json` | `scripts/lns_final.py --base final.json --flow ... --stop-at-gt` | `final_1024.json` | 1.0000 | 1.0000 | 1 h 6 min |

Scores are weighted quality (ground truth = 1.000, lower is better). Steps 3–6 and 8 each use 6 worker processes. The rebuild column is the full run of 10/24–25 (`prime_results/repro_full/`, write-up in `prime_results/repro_full_report.md`): 12 h 39 min in all, every output 100/100 valid, steps 3–6 0.10–0.13 better than the published runs.

Single designs swing by up to ±0.25 between runs of one flow; the best-of over six flows absorbs most of that. Smoke tests on designs 1 and 5 (10/23; design 5 run from the unpacked archive) went through steps 1–7 and gave 1.000, the same as `final_1023.json`. Step 8 was checked on designs 8, 50 and 76 with short rounds (`LNS_ROUNDS=60,60`): it fixed 8 by the cluster shift and moved 76 to its zero-white-space layout (1.041 to 1.017). In the full rebuild, the first run of step 8 ended at 1.0001: design 50's pocket swap stopped at the first anneal that met its pin even though that layout scored worse than the start, so the repair was rejected and no other seed was tried. Since 10/25 it stops only on a result with fewer violations and a lower score; step 8 was rerun and reached 100 at 1.000 (the first output is kept as `final_lns_v1.json`). Step 8 stops a design once its contest cost reaches 1.000 (`--stop-at-gt`), which uses the ground truth to decide where the time goes; every decision to keep a layout uses only our own measures.

### 3. How the published results were assembled

The published files were made over 10/21–10/23 by the chain scripts listed under Scripts, while the code was still changing. `final_1023.json` is `final_1022b.json` (best-of + polish over the six flow files above plus `final_1022.json`) with three fixes applied by hand on 10/23:

- Design 75 was re-solved with the pocket fix (`prime_results/fix75.json`).
- Design 77 was re-picked with the corrected `relative_cost`.
- Designs 50, 77, 80, 81 and 98 got another 60 s refinement with route pushes, kept only where it improved.

All three fixes are now in the code, so the from-scratch rebuild (level 2) includes them.

`final_1024.json` is `final_1023.json` with 17 designs changed by hand-run steps on 10/23–24. Each new layout was kept only if the evaluator found it valid and cheaper (`scripts/merge_layout.py`):

- Designs 77 and 98: the corridor re-slice (10/23 evening, `final_1023b.json`, 1.0030).
- Design 80: window re-partition for its edge pin; design 50: pocket swap (1 of about 35 annealing runs found it); design 8: cluster shift on its zero-white-space layout.
- LNS rounds: 13 designs at 900 s, then rounds of 1,200–2,400 s with up to 4 seeds on the designs still above 1.000; design 76 started from its zero-white-space layout and took six rounds. About 16 CPU-hours in all, 10 of them on design 76.

The batch runs are in `logs/lns_batch_1024*.log`, the layouts in `prime_results/lns_1024/`, and snapshots after each round in `prime_results/final_1024_lns1.json` to `final_1024_lns5.json`. Step 8 of the rebuild runs the same moves and rounds without the hand steps.

`final_1024_wl.json` (10/24) is `final_1024.json` after a wirelength round: LNS for 900 s on every design (seed 20, `prime_results/lns_1024/lns_<id>_s20.json`), each result kept by `merge_layout.py --by-hpwl` (valid, no more soft violations, shorter wirelength). 97 designs got shorter; designs 23 and 73 were replaced by their strict edge-contact repairs (`prime_results/lns_1024/strict/`). Mean HPWL gap −3.13% → −4.06%.

`final_1025.json` (10/25, the current result) is `final_1024_wl.json` with 14 designs taken from the rebuild's `final_lns.json` where its wirelength is shorter (0, 5, 6, 12, 25, 36, 48, 50, 77, 80, 83, 86, 97, 98), each re-scored and carrying the rebuild's own run time. Mean HPWL gap −4.17%; it passes the strict edge-contact and round-trip checks.

## Known limitations and open items

- **Thin margins.** Every design is at 1.000. In `final_1025.json` the closest on wirelength are 76 (0.16% below the ground truth), 29 and 81 (0.4%); 9 designs are less than 1% below it. A different run of the timed searches could land one of them just above it.
- **Search time steered with the ground truth.** The LNS rounds, and step 8 of the rebuild, stop a design once it reaches 1.000. A contest run needs its own stopping rule (e.g. a minimum gain per round); its score hasn't been measured.

- **Edge pins (resolved 10/24).** Designs 50, 77, 80 and 98 were fixed by the corridor, window and pocket moves. The pocket swap is hit and miss (design 50: about 5% of annealing runs, all in the smallest window), so it now spends 12 seeds on that window and stops at the first result with fewer violations and a lower score (10/25; stopping at the first that met the pin cost design 50 its fix in the rebuild's first step-8 run).
- **Edge contact.** The contest check counts any touch, even a sliver of zero area. Designs 23 and 73 of `final_1024.json` met their edge pin only through a 1e-6-high sliver, which the refiner's output rounding can erase. Since 10/24 the refiner requires at least 1e-3 of contact along the edge; 23 and 73 were repaired that way (73 by LNS from its partition + refine layout, since the repair moves couldn't move block 22 onto the edge). The refiner's output step was also fixed: when merging near-equal coordinates breaks a contact or makes blocks overlap, it keeps those blocks' exact shapes (designs 25, 31, 33 and 62 failed that round trip before, so LNS could never improve them).
- **Grid flow** (`prime_grid.py`, `rl/global_alm.py`, `rl/grid_assign.py`). Runs end to end but is rarely valid; the global placement step doesn't converge at zero white space.
- **Pick rule in the Lite repo.** The original Lite solver had the bug fixed here on 10/23 (`contest_relative_cost` added cost ratios instead of gaps); the fix is patched there but not yet measured with a full Lite evaluation.
- **RL fine-tuning.** Anneal-reward tree-builder fine-tuning and the learned kick policy both failed to beat their baselines on held-out designs, so the final flow uses neither. Learning which LNS window to try (3.4% of the 10,691 windows tried won) is the next RL candidate.
- **Thin strips.** Refined layouts often contain very thin soft blocks; the contest has no aspect-ratio rule.
- **Runtime.** The work optimized quality, not runtime: a full 100-design rebuild takes about 13 h (12 h 39 min on 10/24–25). `final_1025.json` records 1,503 s per design on average, mostly the wirelength round.